

```
# =====
# NETFLIX USER ANALYTICS ASSIGNMENT
# =====

# Q1 - Load Dataset and Display First Five Records

import pandas as pd
import numpy as np

df = pd.read_csv("Dataset 2 (1).csv")

print("First 5 Records:")
print(df.head())
```

First 5 Records:

	UserID	Age	Gender	SubscriptionType	WatchHoursPerWeek	DevicesUsed	\
0	1001	22	Female	Basic	23	5	
1	1002	55	Male	Basic	9	4	
2	1003	49	Male	Basic	8	3	
3	1004	39	Female	Premium	19	5	
4	1005	38	Female	Premium	23	5	

	FavoriteGenre	AdClicks	MonthlySpend	SubscriptionRenewed
0	Comedy	13	353	No
1	Drama	14	317	Yes
2	Comedy	16	309	No
3	Drama	45	833	Yes
4	Sci-Fi	24	804	Yes

```
# =====
# PART A : DATASET UNDERSTANDING
# =====

# Q2 - Number of Rows and Columns

print("\nDataset Shape:")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])

# Q3 - Display Column Names

print("\nColumn Names:")
print(df.columns.tolist())

# Q4 - Numerical and Categorical Features

numerical_features = df.select_dtypes(include=['int64', 'float64']).columns
categorical_features = df.select_dtypes(include=['object']).columns

print("\nNumerical Features:")
print(list(numerical_features))

print("\nCategorical Features:")
print(list(categorical_features))

# Q5 - Check Missing Values

print("\nMissing Values:")
print(df.isnull().sum())
```

Dataset Shape:

Rows: 750

Columns: 10

Column Names:

['UserID', 'Age', 'Gender', 'SubscriptionType', 'WatchHoursPerWeek', 'DevicesUsed', 'FavoriteGenre', 'AdClicks', 'MonthlySpend', 'SubscriptionRenewed']

Numerical Features:

['UserID', 'Age', 'WatchHoursPerWeek', 'DevicesUsed', 'AdClicks', 'MonthlySpend']

Categorical Features:

['Gender', 'SubscriptionType', 'FavoriteGenre', 'SubscriptionRenewed']

```
Missing Values:
UserID          0
Age             0
Gender          0
SubscriptionType 0
WatchHoursPerWeek 0
DevicesUsed     0
FavoriteGenre   0
AdClicks        0
MonthlySpend    0
SubscriptionRenewed 0
dtype: int64
```

```
# =====
# PART B : EXPLORATORY DATA ANALYSIS
# =====

# Q6 - Average Age

print("\nAverage Age:")
print(df['Age'].mean())

# Q7 - Average Watch Hours Per Week

print("\nAverage Watch Hours Per Week:")
print(df['WatchHoursPerWeek'].mean())

# Q8 - Average Monthly Spending

print("\nAverage Monthly Spending:")
print(df['MonthlySpend'].mean())

# Q9 - Count Users in Each Subscription Category

print("\nUsers in Each Subscription Category:")
print(df['SubscriptionType'].value_counts())

# Q10 - Percentage of Users Who Renewed Subscription

renewed_percentage = (
    (df['SubscriptionRenewed'] == 'Yes').sum()
    / len(df)
) * 100

print("\nPercentage of Renewed Users:")
print(round(renewed_percentage, 2), "%")
```

```
Average Age:
41.824

Average Watch Hours Per Week:
14.236

Average Monthly Spending:
689.9053333333334

Users in Each Subscription Category:
SubscriptionType
Basic      342
Premium    279
VIP        129
Name: count, dtype: int64

Percentage of Renewed Users:
46.27 %
```

```
# =====
# PART C : DATA PREPARATION
# =====

# Q11 - Convert Categorical Features into Numerical Form

from sklearn.preprocessing import LabelEncoder
```

```

df_encoded = df.copy()

label_encoders = {}

categorical_cols = [
    'Gender',
    'SubscriptionType',
    'FavoriteGenre',
    'SubscriptionRenewed'
]

for col in categorical_cols:
    le = LabelEncoder()
    df_encoded[col] = le.fit_transform(df_encoded[col])
    label_encoders[col] = le

print("\nEncoded Dataset:")
print(df_encoded.head())

# Q12 - Define X and y

X = df_encoded.drop(['SubscriptionRenewed'], axis=1)
y = df_encoded['SubscriptionRenewed']

print("\nFeature Set Shape:", X.shape)
print("Target Shape:", y.shape)

# Q13 - Train Test Split

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))

```

Encoded Dataset:

	UserID	Age	Gender	SubscriptionType	WatchHoursPerWeek	DevicesUsed	\
0	1001	22	0	0	23	5	
1	1002	55	1	0	9	4	
2	1003	49	1	0	8	3	
3	1004	39	0	1	19	5	
4	1005	38	0	1	23	5	

	FavoriteGenre	AdClicks	MonthlySpend	SubscriptionRenewed
0	1	13	353	0
1	2	14	317	1
2	1	16	309	0
3	2	45	833	1
4	5	24	804	1

Feature Set Shape: (750, 9)
Target Shape: (750,)

Training Samples: 600
Testing Samples: 150

```

# =====
# PART D : DECISION TREE CLASSIFICATION
# =====

# Q14 - Train Decision Tree

from sklearn.tree import DecisionTreeClassifier

dt_model = DecisionTreeClassifier(random_state=42)

dt_model.fit(X_train, y_train)

# Q15 - Accuracy

```

```

from sklearn.metrics import accuracy_score

dt_predictions = dt_model.predict(X_test)

dt_accuracy = accuracy_score(y_test, dt_predictions)

print("\nDecision Tree Accuracy:")
print(round(dt_accuracy * 100, 2), "%")

# Q16 - Confusion Matrix

from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, dt_predictions)

print("\nDecision Tree Confusion Matrix:")
print(cm)

```

```

Decision Tree Accuracy:
56.67 %

Decision Tree Confusion Matrix:
[[50 32]
 [33 35]]

```

```

# =====
# PART E : KNN CLASSIFIER
# =====

# Q17 - Train KNN (K = 5)

from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

knn_model = KNeighborsClassifier(n_neighbors=5)

knn_model.fit(X_train_scaled, y_train)

knn_predictions = knn_model.predict(X_test_scaled)

# Q18 - Compare Accuracy

knn_accuracy = accuracy_score(y_test, knn_predictions)

print("\nKNN Accuracy:")
print(round(knn_accuracy * 100, 2), "%")

print("\nAccuracy Comparison")
print("Decision Tree:", round(dt_accuracy * 100, 2), "%")
print("KNN:", round(knn_accuracy * 100, 2), "%")

```

```

KNN Accuracy:
56.67 %

Accuracy Comparison
Decision Tree: 56.67 %
KNN: 56.67 %

```

```

# =====
# PART F : LINEAR REGRESSION
# =====

# Q19 - Train Linear Regression Model

from sklearn.linear_model import LinearRegression

X_reg = df_encoded.drop(['MonthlySpend'], axis=1)
y_reg = df_encoded['MonthlySpend']

```

```

X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg,
    y_reg,
    test_size=0.2,
    random_state=42
)

lr_model = LinearRegression()

lr_model.fit(X_train_reg, y_train_reg)

# =====
# Q20 - Predict Monthly Spending for New User
# =====

new_user = pd.DataFrame({
    'UserID':[2000],
    'Age':[30],
    'Gender':[1],          # Male
    'SubscriptionType':[1], # Example encoded value
    'WatchHoursPerWeek':[20],
    'DevicesUsed':[3],
    'FavoriteGenre':[0],   # Example encoded value
    'AdClicks':[15],
    'SubscriptionRenewed':[1] # Yes
})

predicted_spending = lr_model.predict(new_user)

print("\nPredicted Monthly Spending:")
print("₹", round(predicted_spending[0], 2))

```

Predicted Monthly Spending:
₹ 849.09

```

# =====
# FEATURE IMPORTANCE (Useful for Business Reflection)
# =====

feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': dt_model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by='Importance',
    ascending=False
)

print("\nDecision Tree Feature Importance:")
print(feature_importance)

```

Decision Tree Feature Importance:

	Feature	Importance
8	MonthlySpend	0.217700
0	UserID	0.186622
1	Age	0.171988
4	WatchHoursPerWeek	0.160064
7	AdClicks	0.128323
6	FavoriteGenre	0.057307
5	DevicesUsed	0.054726
2	Gender	0.019920
3	SubscriptionType	0.003350

